

Build: Database Persistence Layer & API Endpoints

Objective

Create a **Database Persistence Layer** that stores all deterministic outputs of EnterprateAI engines and orchestrator, including:

- assessments
- scores
- risk flags
- recommendations
- fragility profiles
- scenario runs
- ASI outputs

Expose **REST API endpoints** for:

- creating assessments
- running scenarios
- retrieving intelligence for frontend, PDF reports, or AI narrative generation

1. Recommended Database Tables

All tables should be relational (PostgreSQL), using UUIDs as primary keys.

A. Assessments

```
CREATE TABLE assessments (  
  id UUID PRIMARY KEY,  
  business_id UUID NOT NULL,  
  engine_type VARCHAR NOT NULL,  
  ruleset_version VARCHAR,
```

```
input_snapshot JSONB,  
output_snapshot JSONB,  
confidence_score NUMERIC,  
confidence_level VARCHAR,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

B. Assessment Scores

```
CREATE TABLE assessment_scores (  
  id UUID PRIMARY KEY,  
  assessment_id UUID REFERENCES assessments(id),  
  score_name VARCHAR NOT NULL,  
  score_value NUMERIC NOT NULL,  
  weight NUMERIC,  
  classification_code VARCHAR  
);
```

C. Risk Flags

```
CREATE TABLE risk_flags (  
  id UUID PRIMARY KEY,  
  assessment_id UUID REFERENCES assessments(id),  
  engine_name VARCHAR,  
  code VARCHAR NOT NULL,  
  severity VARCHAR,  
  explanation TEXT,  
  recommended_action TEXT,  
  priority_score NUMERIC,  
  active_flag BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

D. Recommendations

```
CREATE TABLE recommendations (  
  id UUID PRIMARY KEY,  
  assessment_id UUID REFERENCES assessments(id),  
  source_engine VARCHAR,
```

```
type VARCHAR,  
priority VARCHAR,  
recommendation_text TEXT,  
expected_impact TEXT,  
implementation_difficulty VARCHAR,  
status VARCHAR DEFAULT 'open',  
created_at TIMESTAMP DEFAULT NOW()  
);
```

E. Fragility Profiles

```
CREATE TABLE fragility_profiles (  
  id UUID PRIMARY KEY,  
  business_id UUID NOT NULL,  
  assessment_id UUID REFERENCES assessments(id),  
  fragility_index NUMERIC,  
  classification VARCHAR,  
  cash_fragility NUMERIC,  
  revenue_fragility NUMERIC,  
  customer_fragility NUMERIC,  
  operational_fragility NUMERIC,  
  capacity_fragility NUMERIC,  
  market_fragility NUMERIC,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

F. Scenario Runs

```
CREATE TABLE scenario_runs (  
  id UUID PRIMARY KEY,  
  business_id UUID NOT NULL,  
  scenario_id UUID,  
  base_assessment_id UUID,  
  simulated_state_snapshot JSONB,  
  deltas JSONB,  
  recommended_decision_path JSONB,  
  asi_triggers JSONB,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

2. API Endpoints

A. Create Master Assessment

POST /businesses/:id/assessments/master

Content-Type: application/json

Authorization: Bearer <token>

Body:

```
{
  "financialData": {...},
  "commercialData": {...},
  "operationalData": {...},
  "marketData": {...},
  "customerData": {...},
  "dependencies": [...],
  "metadata": {...}
}
```

Response:

```
{
  "success": true,
  "assessmentId": "<uuid>",
  "output": EnterprateIntelligenceOutput
}
```

B. Run Scenario / ASI

POST /businesses/:id/scenarios/run

Content-Type: application/json

Authorization: Bearer <token>

Body:

```
{
```

```
"scenarioChanges": [  
  {  
    "entityType": "offering",  
    "fieldName": "price",  
    "oldValue": 100,  
    "newValue": 120,  
    "changeDirection": "increase",  
    "changePct": 20  
  },  
  ...  
],  
"metadata": {...}  
}
```

Response:

```
{  
  "success": true,  
  "scenarioRunId": "<uuid>",  
  "output": ScenarioEngineOutput  
}
```

C. Retrieve Assessment / Scenario Results

GET /businesses/:id/assessments/:assessmentId

Authorization: Bearer <token>

Response:

```
{  
  "success": true,  
  "output": EnterpriseIntelligenceOutput  
}
```

GET /businesses/:id/scenarios/:scenarioRunId

Authorization: Bearer <token>

Response:

```
{  
  "success": true,
```

```
"output": ScenarioEngineOutput
}
```

3. Key Implementation Notes

1. **Atomic Transactions:** Ensure all engine outputs for one assessment or scenario are persisted atomically.
2. **Traceability:** Persist trace objects to allow AI narratives to explain calculations.
3. **Versioning:** Include `rulesetVersion`, `engineVersion` in all persisted records.
4. **Soft Delete:** Risk flags and recommendations can be soft-deleted (`active_flag = false`).
5. **JSONB Usage:** Use JSONB for dynamic snapshots (`input_snapshot`, `output_snapshot`, `simulated_state_snapshot`).
6. **MVP Priority:** Start with assessment creation + scenario run endpoints; retrieval endpoints for frontend and PDF generation.

4. Future Enhancements

- Batch insert multiple scenario simulations
- Async job queue for heavy simulations
- WebSocket notifications for frontend updates
- Audit logs for each persistence action
- API pagination for historical assessments and scenario runs

5. Codex Prompt for Implementation

Build the EnterprateAI Database Persistence Layer and API endpoints.

Requirements:

1. PostgreSQL tables for:
 - assessments

- assessment_scores
- risk_flags
- recommendations
- fragility_profiles
- scenario_runs

2. REST API endpoints:

- POST /businesses/:id/assessments/master
- POST /businesses/:id/scenarios/run
- GET /businesses/:id/assessments/:assessmentId
- GET /businesses/:id/scenarios/:scenarioRunId

3. Persist engine outputs atomically

4. Include trace objects and versioning

5. Use JSONB for dynamic snapshots

6. Implement soft delete for risks/recommendations

7. Provide example request/response objects for frontend, AI narrative, and PDF reports

8. Add unit tests for database persistence and API endpoints